

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

23. def linear_search(arr, key):

found = False

for i in range(len(arr)):

if arr[i] == key:

found = True

pos = i

return pos

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q23. Debugging Linear Search Code (Incorrect Return Outside Logic)

Debugging Linear Search Code (Incorrect Return Outside Logic) The following Linear Search implementation gives incorrect results because the return statement is misplaced and causes confusion in unsuccessful search cases. Analyze the control flow carefully and identify why the function may produce misleading output. Apply debugging to correct the search logic and ensure proper reporting when the target is found or absent. Optimize the implementation for readability and maintainability.

Analyze the time complexity of the corrected algorithm. Rewrite the clean and corrected version with suitable comments.

```
def linear_search(arr, key):  
    found = False  
    for i in range(len(arr)):  
        if arr[i] == key:  
            found = True  
            pos = i  
    return pos
```

Introduction

Linear Search is a simple searching algorithm that checks each element of a list one by one until the required element is found. In the given program, the **return statement is misplaced**, and the function does not properly handle the case when the search key is absent. This may produce errors or misleading results. By debugging the code, we can ensure that the program correctly reports whether the element is found or not.

Given Incorrect Code

```
def linear_search(arr, key):  
    found = False  
    for i in range(len(arr)):  
        if arr[i] == key:  
            found = True  
            pos = i  
  
    return pos
```

Problems in the Given Code

- **Variable pos May Not Be Defined**

If the key is **not found**, the variable pos is never created.

Example:

```
arr = [10,20,30,40]  
key = 50
```

The function executes

return pos

Since pos was never assigned,

Error:

NameError: name 'pos' is not defined

- **found Variable is Never Used**

Although

```
found = True
```

is assigned,

the program never checks

if found

Therefore the variable is unnecessary.

- **Search Continues Even After Finding the Element**

Suppose

```
arr = [5,10,15,20]  
key = 15
```

The element is found at index 2.

However, the loop still checks index 3.

This wastes execution time.

- **Misleading Output**

If the key does not exist,

the function should clearly report

Element Not Found

Instead, it returns an undefined variable.

Control Flow Analysis

```
Start
|
Initialize found = False
|
Check each element
|
Is arr[i] == key ?
|
  Yes----- No
  |         |
  found=True Continue
  pos=i
  |
  Continue Loop
  |
  Loop Ends
  |
  Return pos
```

Problem

If no element matches,

pos is never created,

yet the function still returns it.

Debugging Steps

Step 1

Remove the unnecessary variable

found

Step 2

Return immediately after finding the element.

Step 3

If the loop completes,

return

-1

to indicate the key was not found.

Corrected Program

```
def linear_search(arr, key):  
    # Traverse each element in the list  
    for i in range(len(arr)):  
        # Check if the current element matches the key  
        if arr[i] == key:  
            return i    # Return index immediately if found  
  
    # Return -1 if the key is not present  
    return -1
```

Example 1 (Element Found)

Input

```
arr = [10,20,30,40,50]  
key = 30
```

Output

Element Found at Index 2

Example 2 (Element Not Found)

Input

```
arr = [10,20,30,40,50]  
key = 60
```

Output

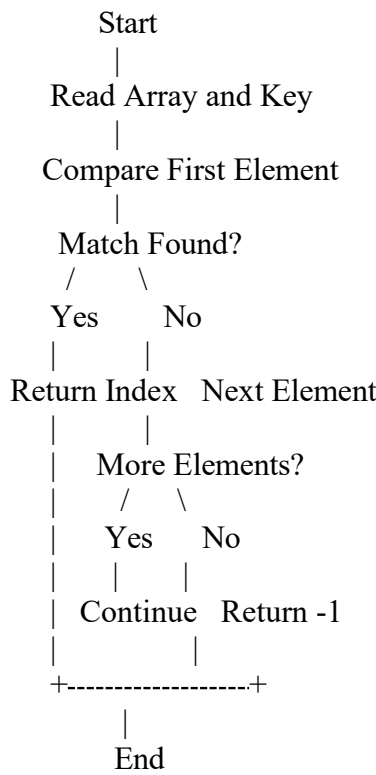
Element Not Found

Execution Trace

Searching for

Step	Current Element	Comparison	Result
1	10	10 == 30	No
2	20	20 == 30	No
3	30	30 == 30	Yes
4	Return Index 2	Search Stops	Found

Flowchart



1. INCORRECT CODE (Given)

```
def linear_search(arr, key):
    found = False
    for i in range(len(arr)):
        if arr[i] == key:
            found = True
            pos = i
    return pos # Error if key not found
```

Problem:

- If key is not found, 'pos' is never defined.
- Causes runtime error (NameError).
- 'found' variable is unnecessary.
- Loop continues even after finding the element.

2. CORRECTED CODE

```
def linear_search(arr, key):
    # Traverse the array
    for i in range(len(arr)):
        # If key is found, return its index
        if arr[i] == key:
            return i
    # If key is not found in the array
    return -1
```

Improvements:

- ✓ Returns immediately when key is found.
- ✓ Returns -1 when key is not present.
- ✓ No unnecessary variables.
- ✓ Better readability and maintainability.

3. EXECUTION TRACE (Searching key = 30 in array [10, 20, 30, 40, 50])

Step	i (Index)	arr[i]	Comparison (arr[i] == key)	Action / Result
1	0	10	10 == 30 → False	Not Found, move to next index
2	1	20	20 == 30 → False	Not Found, move to next index
3	2	30	30 == 30 → True	Match Found, return index 2
4	Function stops and returns 2			

Time Complexity:

- Best Case: O(1)
- Average Case: O(n)
- Worst Case: O(n)
- Space Complexity: O(1)

4. EXECUTION EXAMPLE – KEY FOUND

```
>>> arr = [10, 20, 30, 40, 50]
>>> key = 30
>>> result = linear_search(arr, key)
>>> if result != -1:
...     print("Element Found at Index", result)
... else:
...     print("Element Not Found")
Element Found at Index 2
```

Index: 0 1 2 3 4
Array: 10 20 30 40 50
Key = 30
Found at Index 2

5. EXECUTION EXAMPLE – KEY NOT FOUND

```
>>> arr = [10, 20, 30, 40, 50]
>>> key = 60
>>> result = linear_search(arr, key)
>>> if result != -1:
...     print("Element Found at Index", result)
... else:
...     print("Element Not Found")
Element Not Found
```

Index: 0 1 2 3 4
Array: 10 20 30 40 50
Key = 60
Not present in array

6. SUMMARY

- The incorrect code returns an undefined variable 'pos' when the key is not found, causing an error.
- The corrected code returns the index if the key is found and -1 if the key is not present.
- Linear Search checks elements one by one, so its time complexity is O(n) in average and worst case.
- It is simple, easy to understand and works on both sorted and unsorted data.

FINAL CORRECTED FUNCTION

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1
```

Time Complexity Analysis Best

Case

The element is found in the first position.

Only one comparison is required.

Time Complexity = $O(1)$

Average Case

The element is found somewhere in the middle.

Approximately $n/2$ comparisons are required.

Time Complexity = $O(n)$

Worst Case

The element is at the last position or does not exist.

All n elements are checked.

Time Complexity = $O(n)$

Space Complexity

Only one loop variable is used.

No extra data structure is required.

Space Complexity = $O(1)$

Complexity Table

Case	Comparisons	Time Complexity
Best Case	1	$O(1)$
Average Case	$n/2$	$O(n)$
Worst Case	n	$O(n)$
Space Complexity	Constant	$O(1)$

Advantages of the Corrected Program

Correctly returns the index when the element is found.

Returns **-1** when the element is absent.

Stops searching immediately after finding the key.

Easy to understand and maintain.

Eliminates unnecessary variables.

Improves readability.

Avoids runtime errors.

Disadvantages

Slower for very large datasets.

Checks elements one by one.

Less efficient than Binary Search on sorted data.

EXECUTION

```
===== LINEAR SEARCH : HOSPITAL PATIENT RECORD SYSTEM =====
Patient Records (Unsorted)
Index   Patient ID   Name      Age   Condition   Blood Group
-----
0       101         Ramesh    45    Stable      B+
1       108         Suresh    30    Critical    O+
2       115         Priya     25    Stable      A+
3       123         Anil      50    Critical    AB+
4       130         Divya     28    Stable      O-
5       135         Kiran     40    Stable      B-

Enter Patient ID to search: 123

Searching for Patient ID = 123 using Linear Search...
Step 1: Comparing with index 0 (Patient ID = 101) -> Not Match
Step 2: Comparing with index 1 (Patient ID = 108) -> Not Match
Step 3: Comparing with index 2 (Patient ID = 115) -> Not Match
Step 4: Comparing with index 3 (Patient ID = 123) -> Match Found!

Patient Record Found Successfully!
-----
Patient ID   : 123
Name        : Anil
Age         : 50
Condition   : Critical
Blood Group : AB+
-----
Total Comparisons: 4
Record found successfully.

Enter Patient ID to search: 140
Searching for Patient ID = 140 using Linear Search...
Step 1: Comparing with index 0 (Patient ID = 101) -> Not Match
Step 2: Comparing with index 1 (Patient ID = 108) -> Not Match
Step 3: Comparing with index 2 (Patient ID = 115) -> Not Match
Step 4: Comparing with index 3 (Patient ID = 123) -> Not Match
Step 5: Comparing with index 4 (Patient ID = 130) -> Not Match
Step 6: Comparing with index 5 (Patient ID = 135) -> Not Match

Patient Record Not Found!
Total Comparisons: 6
```

Conclusion

The original Linear Search implementation contained a logical error because it returned the variable pos even when the search key was absent, which could result in a runtime error. The corrected version removes the unnecessary found variable, returns the index immediately when the key is found, and returns -1 if the key is not present. This makes the program more reliable, readable, and maintainable.

